📌 FINAL PROJECT GUIDE

Program: AI/ML/DS Fellowship 1 **– Solo Project**

Project Type: Individual
Deadline: 2-4 Weeks
Total Marks: (Project Implementation: 40 | Reports: 20 | Medium Article: 10 | Interview: 50)

This project will demonstrate your ability to develop and deploy machine learning models and integrate them into a practical application.

✅ **Project Goal:** A **fully functional AI-powered web application** — trained, deployed, documented, and hosted online, showcasing your complete data science workflow from data collection to live deployment.

Your final project should clearly demonstrate your ability to apply **Data Science from data to deployment i.e** your complete **data science workflow**:

Data → Model → Deployment → Documentation → Presentation

Note: Make sure to create a Project Plan/Timeline to track your progress, starting from Monday i.e 23 March to 18 April

1. PROJECT OVERVIEW

Develop an **AI-powered predictive or analytical system** that solves/addresses a **real-world problem** in your chosen domain (e.g., finance, healthcare, energy, e-commerce, environment,education etc.).

Your project must demonstrate your ability to:

1. Build **at least two ML/DL models**. (compare or integrate).
2. **Deploy** your trained model(s) using Pickle/TensorFlow SavedModel.
3. **Create APIs (Flask/FastAPI)** or **interactive UI (Streamlit)** to connect and use models.
4. Ensure **the deployed model is accessible via a cloud service**. **Host** the project on a cloud service (Streamlit Cloud, Azure, AWS, etc.).
5. Document the entire **workflow and results** in a **Medium article** and a **final report**.

- **Deliverables:**

    1. **GitHub Repository –** clean, organized, with code, README, documentation.
    2. **Working Demo/Deployed Application Link (API/UI) –** Streamlit Cloud / Azure / Render.

3. **Medium Article –** documenting process, project workflow, visuals, results and learnings.

4. **Final Report + Presentation Slides(5–10 slides) –** PDF format**.**

5. **Interview (explain and defend project)**.

Guidelines

1. **Individual project** (solo).

2. **Document entire workflow**: data →methods→ workflow→ results→ deployment.

3. **UI requirement**:

   - Can use **Streamlit** to build a web app, OR

   - Flask/FastAPI for REST APIs.

4. **Deployment options**:

   - **Streamlit Cloud** (easy)

   - **Azure Web App** (with GitHub Actions CI/CD)

   - **AWS / Render / Heroku** (optional).

2. STEP-BY-STEP WORKFLOW

1 – Project Setup & Problem Definition

Clearly state what real-world problem your system aims to solve and why it matters.

1. Pick a **domain**: healthcare, finance, energy, education, retail, etc.

2. Define **problem statement** (clear, measurable, real-world impact). Example: "Predict student dropout risk based on academic, demographic, and behavioral data."

3. Set up project repo (GitHub).

   Example Folder structure:

```
project_name/
├── data/
├── notebooks/
├── models/
├── app/ (Flask or Streamlit files)
├── requirements.txt
├── README.md
└── reports/
```

README.md must contain:

- Project title & short description
- Problem statement
- Dataset source
- Model details
- Deployment link
- Medium article link
- Instructions to run locally
- License
- Other required stuff (Workflow/Performance Visuals)

2 – Data Acquisition

- Identify and collect relevant datasets (**open**-source or **self**-collected).
- **Collect dataset(s**). Sources: Kaggle, UCI ML Repo, government portals, APIs.

3 – Data Preparation & EDA

- Clean data: handle missing values, duplicates, outliers.
- Perform **EDA** (charts, correlations, distributions).
- Document insights.

Deliverables:

- Notebook with EDA plots.
- Short summary of key insights.

4 – Feature Engineering

- Create new meaningful features (e.g., ratios, time-based features).
- Encode categorical variables.
- Scale/normalize numerical features.
- Split into train/test sets.

Deliverable:

- Processed dataset ready for ML.

4 – Model Development

- Train at least **two models**:

    - ML model (e.g., Logistic Regression, Random Forest, XGBoost).
    - DL model (e.g., ANN, CNN, LSTM depending on domain).
- Compare performance. (evaluate performance using metrics like accuracy, precision, recall, F1-score, RMSE, etc.)
- Fine-tune hyperparameters. Save the best-performing models

Deliverables:

- Performance metrics (accuracy, precision, recall, F1, AUC, RMSE depending on task).
- Saved models (.pkl or TensorFlow format).

5 – Model Deployment & API

Option 1: Flask / FastAPI

- Build a **Flask/FastAPI app**:
    - Create API endpoints for predictions.
- If needed, integrate **2 models** (ensemble or hybrid).
- Deploy locally first, then to **Heroku / Render / AWS / PythonAnywhere**.

Deliverables: Running API. Code with endpoints.

Option 2: Streamlit UI (Recommended)

- Build a web UI for predictions
- **Hosting Options:**
    1. **Streamlit Cloud:** push GitHub repo → connect to Streamlit Cloud → auto-deploy.
    2. **Azure Web App** via **GitHub Actions CI/CD** (see below).

🌐 DEPLOYMENT & CI/CD INTEGRATION

Hosting Options:

- **A. Streamlit Cloud: Push repo → Connect Streamlit → Deploy automatically.**
- **B. Azure Web App: Use GitHub Actions CI/CD (auto redeploy on push).**

Option A. Using Streamlit Cloud

1. Push project to GitHub.

2. Go to https://share.streamlit.io.
3. Connect your repo.
4. Deploy automatically — app available at https://your-app-name.streamlit.app.

Option B. Using Azure + GitHub Actions CI/CD

Example workflow for continuous deployment:

```yaml
name: Deploy to Azure Web App

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v2
    - name: Set up Python
      uses: actions/setup-python@v2
      with:
        python-version: '3.10'
    - name: Install dependencies
      run: pip install -r requirements.txt
    - name: Deploy to Azure
      uses: azure/webapps-deploy@v2
      with:
        app-name: "your-app-name"
        publish-profile: ${{ secrets.AZURE_PUBLISH_PROFILE }}
        package: .
```

6 – Reporting & Documentation

Report Outline

A. Write **Medium Article**: explain Introduction (problem + motivation, dataset, models, results, and deployment, key learnings, conclusion.

B. Prepare **Technical Report (PDF,** in ***reports/*** folder**)**:

1. **Cover Page** (Title, Name, Date, Fellowship)
2. **Abstract** (150–200 words)

3. **Introduction & Problem Statement**
4. Data Collection / **Dataset Description**
5. **Data Preprocessing & EDA**
6. **Feature Engineering**
7. **Modeling Approach (**Models (ML/DL)**)**
8. **Results & Evaluation Metrics**
9. **Deployment Process** (API/Streamlit + Hosting details)
10. **CI/CD** (if applied)
11. **Business/Real-world Impact**
12. **Challenges & Limitations**
13. **Conclusion** & **Future Work**
14. **References**

C. Create **slides** for presentation (5–10 slides). Problem → Approach → Models → Results → Impact, etc.

Deliverables:

- Medium article link.
- Report (PDF).
- Presentation slides.

7 – Final Touches & Interview Prep

- Test all code.
- Finalize repo README and other docs.
- Practice explaining:
    1. Why this problem?
    2. Why these models?
    3. What challenges faced?
    4. How deployment works?
    5. How this project can scale?

---

3. References & Learning Resources

- [Creating Powerful Presentations]
- [Streamlit Documentation]

- [MS Azure Apps Services Documentation]
- [GitHub Actions for CI/CD]
- [Flask Documentation]
- [AWS for ML Deployment]
- [(Check GCR)]

🗓️ 4-WEEK EXECUTION PLAN (Reference)

Note: Create a MONTHLY Project Plan/Timeline starting from Monday i.e *23 March to 18 April 2026*

WEEK 1: Model Development Phase

Day	Tasks	Deliverables
Day 1 (Kickoff)	Define the **problem statement** and objectives. Collect dataset(s). Create GitHub repo and project folder structure.	Project proposal + GitHub repo setup
Day 2 (Data Prep)	Perform data cleaning, preprocessing, feature engineering.	Clean dataset + preprocessing notebook
Day 3 (EDA)	Perform **exploratory data analysis (EDA)** — visualizations, outlier detection, correlations.	EDA report (graphs, insights)
Day 4 (Model 1)	Implement **Model 1 (ML)** – e.g., Linear Regression, Random Forest, or XGBoost.	Trained model + saved .pkl
Day 5 (Model 2)	Implement **Model 2 (DL)** – e.g., LSTM, CNN, ANN, etc.	Trained model + saved .h5
Day 6 (Evaluation)	Compare performance of both models using metrics (accuracy, F1-score, RMSE). Tune hyperparameters.	Comparison table + best model selected
Day 7 (Integration)	Integrate both models into a unified pipeline. Prepare prediction function.	Final notebook + integrated model

WEEK 2: Deployment & Documentation Phase

Day	Tasks	Deliverables
Day 8 (API/UI Setup)	Choose between **Flask (API)** or **Streamlit (UI)**. Build a simple interface for input + prediction output.	Working local app
Day 9 (Communication)	If using Flask, create **API endpoints** for communication between ML and DL models.	Flask API /predict working locally
Day 10 (Deployment Prep)	Containerize (if needed). Prepare requirements.txt and deployment config.	Ready-to-deploy project
Day 11 (Deployment)	Deploy using one of the following: ◆ Streamlit Cloud (recommended) ◆ Azure Web App (via CI/CD) ◆ AWS Sagemaker/EC2	Live deployed app link
Day 12 (CI/CD Setup)	Configure **GitHub Actions** for automatic deployment to Streamlit/Azure.	.github/workflows/deploy.yml working
Day 13 (Documentation)	Write **Medium article** and prepare **final report + slides**.	Medium article + PDF report
Day 14 (Testing & Submission)	Test entire pipeline. Finalize GitHub repo with README, app link, article link, and report.	Final submission package

Sample Project 1 (for Reference)

Title: AI-Powered Student Dropout Predictor

Author Name: Moneka

Problem Statement

Many universities face high student dropout rates. Identifying at-risk students early can help with timely interventions.

Workflow

1. **Dataset:** UCI Student Performance Dataset + external socio-economic data.
2. **EDA:** Found that study time, parental education, and absences strongly influence dropout risk.
3. **Models Used:**
    - Random Forest (ML)
    - ANN (DL, 3 hidden layers with ReLU + dropout).
4. **Best Model:** ANN with 87% accuracy, F1-score 0.83.
5. **Deployment:** Flask API with two endpoints:
    - /predict_rf → Random Forest
    - /predict_ann → ANN

Output Example (JSON):

```
 {
 "student_id": 123,
 "RF_prediction": "Dropout",
 "ANN_prediction": "Continue",
 "confidence_scores": {
  "RF": 0.71,
  "ANN": 0.83
 }
}
```

Deliverables

- **Medium Article:** "Predicting Student Dropout with ML/DL"
- **GitHub Repo:** (with notebooks, app, models, docs).
- **Report (PDF):** Covers dataset, methodology, models, evaluation, deployment.
- **Slides:** Problem → Approach → Models → Results → Impact.

Author: Moneka M

📍 *Buildables Data Science Fellowship 1 – Final Project*

🔗 **Medium Article:** [link]

🔗 **LinkedIn Article:** [link]

💻 **GitHub Repo:** [github.com/yourusername/yourproject]

🌐 **Deployed App:** [streamlit.app link]

Sample Project 2 (for Reference)

Title: Intelligent Energy Consumption Predictor

Author: [Your Name]

Problem Statement:

Develop an AI-powered system to predict building energy consumption based on historical usage, weather forecasts, and building characteristics.

Major Components:

1. **Energy Consumption Prediction (ML)** – ARIMA or LSTM for time-series forecasting.
2. **Weather Forecast Integration (DL)** – CNN to analyze weather patterns.
3. **API Integration (Flask)** – Communicate between models, fetch forecasts, and provide results.
4. **Deployment (Streamlit or Azure)** – Cloud-hosted web UI or API.

Deliverables:

- Working web app hosted on Streamlit Cloud.
- GitHub repository with full documentation.
- Medium article covering workflow.
- Report and presentation slides.

Guidelines

1. **Individual project — each fellow must build their own.**
2. **Must solve a real-world problem in any domain (healthcare, finance, environment, etc.).**
3. **Must include model documentation, API endpoints/UI, and deployment link.**
4. **All work should be version-controlled on GitHub.**
5. **Document everything in a Medium article and final report.**

✅ Final Deliverables Checklist

✅ GitHub Repository (public, with documentation)

- **Source code**
- **Model files (.pkl or SavedModel)**
- **Requirements.txt**
- **README with setup and deployment instructions**

✅ Deployed Web App

- **Working Flask/Streamlit App**
- **Live Deployment Link (Streamlit Cloud / Azure / Render / Heroku / Pythonanywhere)**

✅ Medium Article

- **Explaining workflow, results, and deployment process, etc**

✅ Final Report (PDF)

- **Include: (Structure given above)**

✅ Slides (PPT)

- **5–10 slides for project presentation/interview**

✅ Monthly Execution Plan (XLS)

- **Create a Project Plan/Timeline starting from Monday i.e 13 Oct to 15 Nov.**

✅ Interview

- **Explain problem, data, models, and deployment live**
- **Prepare for your final assessment test/interview**

🎓 FINAL LEARNING OUTCOMES

By completing this Solo Project, learners will be able to:

1. Problem-Solving & Research Skills

- Identify and define a real-world problem that can be addressed using AI/ML solutions.
- Research and select suitable datasets, frameworks, and deployment tools.

2. Data Handling & Analytical Skills

- Perform data collection, cleaning, preprocessing, and feature engineering.
- Conduct Exploratory Data Analysis (EDA) to extract meaningful insights.
- Understand the importance of data quality in model performance.

3. Model Development & Evaluation

- Build, train, and compare multiple ML and DL models.
- Apply techniques like hyperparameter tuning, cross-validation, and performance optimization.
- Evaluate models using appropriate metrics such as accuracy, precision, recall, F1-score, RMSE, etc.

4. Model Deployment Skills

- Learn to serialize and deploy models using TensorFlow, Pickle, or ONNX formats.
- Implement and test Flask/FastAPI endpoints or Streamlit-based user interfaces.
- Host deployed applications on Streamlit Cloud, Azure, AWS, or Render.
- Understand CI/CD pipelines (GitHub Actions) for automated deployment workflows.

5. Software Development & Integration

- Apply principles of modular coding, version control (Git/GitHub), and project structuring.
- Integrate multiple models or APIs for collaborative system communication.
- Implement logging and monitoring for deployed applications.

6. Technical Documentation & Communication

- Prepare professional technical documentation and project reports.
- Write a detailed Medium article explaining methodology, insights, and results.
- Present and defend the project in a technical interview or viva, demonstrating both technical depth and practical understanding.

7. Professional & Applied Skills

- Gain hands-on experience with end-to-end AI/ML project lifecycle — from data to deployment.
- Develop an understanding of scalability, maintainability, and ethical AI considerations.
- Build a portfolio-ready project showcasing full-stack data science capabilities.